

Ejercicio 1

ALGORITMO:

FCFS-FIFO

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	8		
P2	1	4		
P3	2	2		
P4	3	1		
4		15		
TotalProcesos		Total Ciclos		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
P1	X	X	X	X	X	X	X	X																								
P2		E	E	E	E	E	E	E	X	X	X	X																				
P3			E	E	E	E	E	E	E	E	E	X	X																			
P4				E	E	E	E	E	E	E	E	E	X																			
En Ejecución	P1	P1	P1	P1	P1	P1	P1	P2	P2	P2	P3	P3	P4																			
Cola de procesos Preparados		P2	P2	P2	P2	P2	P2																									
			P3	P3	P3	P3	P3	P3	P3	P3	P3																					
				P4	P4	P4	P4	P4	P4	P4	P4	P4	P4																			

SIF

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	8		
P2	1	4		
P3	2	2		
P4	3	1		
4		15		
TotalProcesos		Total Ciclos		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P1	X	X	X	X	X	X	X	X																							
P2		E	E	E	E	E	E	E	E	E	X	X	X																		
P3			E	E	E	E	E	E	E	X	X																				
P4				E	E	E	E	E	X																						
En Ejecución	P1	P1	P1	P1	P1	P1	P1	P4	P3	P3	P2	P2	P2																		
Cola de procesos Preparados				P3	P3	P3	P3	P3																							
					P4	P4	P4	P4	P4																						

Throughput (Rendimiento)	0,266666667
Nº de cambios de contexto	3
TIEMPO MEDIO ESPERA (WT)	7
TIEMPO MEDIO RETORNO (TAT)	10,75
TIEMPO MEDIO RESPUESTA (RT)	7

Tiempo Espera	Tiempo Respuesta
0	8
7	11
10	12
11	12

Throughput (Rendimiento)	0,266666667
Nº de cambios de contexto	3
TIEMPO MEDIO ESPERA (WT)	5,5
TIEMPO MEDIO RETORNO (TAT)	9,25
TIEMPO MEDIO RESPUESTA (RT)	5,5

Tiempo Espera	Tiempo Respuesta
0	8
10	14
7	9
5	6

Tareas:

Calcula y compara: FCFS y SJF (no expropiativo).

- **FCFS (FIFO):** Es el más **justo** porque atiende a los procesos en el estricto orden de llegada, lo que lo hace simple de implementar. Sin embargo, su **ineficiencia** se debe al "efecto convoy", donde un proceso largo retrasa a todos los demás.
- **SJF (Shortest Job First):** Es el más **eficiente** porque garantiza el **tiempo medio de espera mínimo** para un conjunto de procesos. Esto se logra al ejecutar siempre la ráfaga más corta. Su coste es la **pérdida de equidad** y el riesgo de inanición para los procesos largos.

Obtén WT , TA , R , Throughput, y nº de cambios de contexto.

Métrica Promedio / Total	FCFS	SJF (No Expropiativo)
WT (Espera Promedio)	7.0	5.5
TAT (Retorno Promedio)	10.75	9.25
RT (Respuesta Promedio)	7.0	5.5
Throughput (Rendimiento)	0.266 procesos/u.t.	0.266 procesos/u.t.
Nº Cambios de Contexto	3	3

Concluye: ¿cuál minimiza el tiempo de espera? ¿Cuál es más justo?

El algoritmo **FCFS (First-Come, First-Served)** es el más **justo** (equitativo). FCFS garantiza equidad al respetar el **estricto orden de llegada (FIFO)**, mientras que SJF sacrifica la justicia al priorizar la duración, lo que genera riesgo de **inanición** para los procesos largos.

El algoritmo **SJF (Shortest Job First)** minimiza el tiempo de espera promedio (5.5s), lo que lo convierte en el algoritmo **óptimo para la eficiencia**. Logra esto al asignar la CPU a la tarea con el menor tiempo del **menor tiempo de CPU**, liberando así la cola más rápidamente.

Ejercicio 2 – FCFS vs RR (quantum = 2)

ALGORITMO:

Ejercicio 2 – FCFS vs RR (quantum = 2)

FCFS-FIFO

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	6		
P2	1	3		
P3	2	4		
3		13		
Total Procesos		TOTAL CICLOS		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P1	X	X	X	X	X	X																									
P2		E	E	E	E	E	X	X	X																						
P3			E	E	E	E	E	E	X	X	X	X																			
En Ejecución	p1	p1	p1	p1	p1	p1	p2	p2	p2	p3	p3	p3	p3																		
Cola de procesos							p2	p2	p2	p2																					
Preparados							p3	p3	p3	p3	p3	p3																			

RR

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	6		2
P2	1	3		
P3	2	4		
3		13		
Total Procesos		TOTAL CICLOS		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P1	X	X	E	E	E	E	X	X	E	E	X	X																			
P2		E	X	X	E	E	E	X																							
P3			E	E	X	X	E	E	E	X	X																				
En Ejecución	p1	p1	p2	p2	p3	p3	p1	p1	p2	p3	p3	p1	p1																		
Cola de procesos							p2	p3	p3	p1	p1	p2	p2	p3	p1	p1															
Preparados							p1	p1	p2	p2	p3	p3	p1																		

Throughput (Rendimiento)	0,230769231
Nº de cambios de contexto	3
TIEMPO MEDIO ESPERA (WT)	4
TIEMPO MEDIO RETORNO (TAT)	8,333333333
TIEMPO MEDIO REPUESTA (RT)	4

Tiempo Espera	Tiempo Respuesta	Coste primera ejecucion
0	6	0
5	8	5
7	11	7

Throughput (Rendimiento)	0,230769231
Nº de cambios de contexto	6
TIEMPO MEDIO ESPERA (WT)	5,666666667
TIEMPO MEDIO RETORNO (TAT)	10
TIEMPO MEDIO REPUESTA (RT)	1

Tiempo Espera	Tiempo Respuesta	Coste primera ejecucion
7	13	0
5	8	1
5	9	2

Objetivo: Comparar equidad y tiempo de respuesta.

Pregunta: ¿cuál algoritmo sentiría “más fluido” el usuario si cada proceso fuera una aplicación interactiva?

El algoritmo que se sentiría **más fluido** es **Round Robin (RR)**.

Justificación:

Interactividad: La fluidez se mide por el **Tiempo de Respuesta (RT)**. RR logra un RT de solo **1.0**, mientras que FCFS es 4.0.

Principio RR: RR es un algoritmo expropiativo que, al dar turnos rápidos (q=2), asegura que todos los procesos interactivos reciban la CPU rápidamente, haciendo que las aplicaciones parezcan responder de forma instantánea.

Compromiso: Este excelente tiempo de respuesta tiene un coste: hay **más cambios de contexto** (6 en RR frente a 3 en FCFS), lo que añade sobrecarga al sistema, pero mejora drásticamente la percepción del usuario.

Ejercicio 3 – FCFS vs SRTF

Objetivo:

1. Simula ambos algoritmos.

PLANIFICACIÓN DE PROCESOS

ALGORITMO: Ejercicio 3 – FCFS vs SRTF

FCFS-FIFO

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	10		
P2	3	4		
P3	5	2		
P4	6	6		
4		22		
Total Procesos		TOTAL CICLOS		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P1	X	X	X	X	X	X	X	X	X	X																					
P2				E	E	E	E	E	E	E	X	X	X	X																	
P3						E	E	E	E	E	E	E	E	X	X																
P4							E	E	E	E	E	E	E	E	E	X	X	X	X	X	X										

En Ejecución p1 p1 p1 p1 p1 p1 p1 p1 p1 p1 p1 p2 p2 p2 p2 p2 p2 p3

2. Calcula todas las métricas.

FCFS-FIFO

Throughput (Rendimiento)	0,181818182
Nº de cambios de contexto	3
TIEMPO MEDIO ESPERA(WT)	6,5
TIEMPO MEDIO RETORNO(TAT)	12
TIEMPO MEDIO REPUESTA(RT)	6,5

SRTF

Throughput (Rendimiento)	0,181818182
Nº de cambios de contexto	4
TIEMPO MEDIO ESPERA(WT)	4,25
TIEMPO MEDIO RETORNO(TAT)	9,75
TIEMPO MEDIO REPUESTA(RT)	1,25

3. Discute cómo cambia $W\bar{T}$ y $R\bar{T}$ cuando el sistema puede interrumpir procesos.

La capacidad de interrumpir procesos mejora drásticamente tanto la eficiencia total (menor $W\bar{T}$) como la interactividad (menor $R\bar{T}$) del sistema.

4. ¿Qué pasa con los procesos largos?

El proceso largo paga el precio de la eficiencia y la interactividad del sistema². El TAT de P1 se duplica con creces (de 10 a 22) para que P2, P3 y P4 puedan ejecutarse rápidamente. En sistemas con procesos muy largos, esto puede llevar a la **inanición (starvation)**, donde P1 podría no terminar nunca si continúan llegando procesos cortos.

Ejercicio 4 – SJF vs RR (quantum = 3)

PLANIFICACIÓN DE PROCESOS

ALGORITMO:

Ejercicio 4 – SJF vs RR (quantum = 3)

SJF

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	9		
P2	1	5		
P3	2	3		
P4	3	7		
4		24		
Total Procesos		TOTAL CICLOS		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P1	X	X	X	X	X	X	X	X	X																						
P2		E	E	E	E	E	E	E	E	E	E	E	X	X	X	X	X														
P3			E	E	E	E	E	E	E	E	X	X	X																		
P4				E	E	E	E	E	E	E	E	E	E	E	E	E	X	X	X	X	X	X	X	X							

En Ejecución: p1

Cola de procesos: p2

Preparados: p3

RR

PROCESO	CICLO DE	NÚMERO DE	PRIORIDAD	QUANTUM
P1	0	9		3
P2	1	5		
P3	2	3		
P4	3	7		
4		24		
Total Procesos		TOTAL CICLOS		

RELLENA LA SIGUIENTE TABLA CON X (Ejecución) Y E (Espera)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
P1	X	X	X	E	E	E	E	E	E	E	E	E	X	X	X	E	E	E	E	E	E	X	X	X							
P2		E	E	X	X	X	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E
P3			E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E	E
P4				E	E	E	E	E	E	E	E	E	E	E	E	E	E	X	X	X	E	E	E	E	X						

En Ejecución: p1

Cola de procesos: p2

Preparados: p3

Throughput (Rendimiento)	0,166666667
Nº de cambios de contexto	3
TIEMPO MEDIO ESPERA(WT)	8
TIEMPO MEDIO RETORNO(TAT)	14
TIEMPO MEDIO RESPUESTA(RT)	8

Tiempo Espera	Tiempo Respuesta	Coste primera ejecución
9	0	0
5	11	11
3	7	7
7	14	14

Throughput (Rendimiento)	0,166666667
Nº de cambios de contexto	8
TIEMPO MEDIO ESPERA(WT)	10,75
TIEMPO MEDIO RETORNO(TAT)	16,75
TIEMPO MEDIO RESPUESTA(RT)	3

Tiempo Espera	Tiempo Respuesta	Coste primera ejecución
9	14	0
5	11	2
3	4	4
7	14	6

Objetivo:

1. Calcula todas las métricas.

2. Analiza: ¿qué algoritmo favorece los procesos cortos?

El algoritmo que más favorece a los procesos cortos (P2 con 5s y P3 con 3s) es SJF.

Justificación: En SJF, P3 (el más corto) solo tiene que esperar 9 segundos (a que P1 termine) y luego se ejecuta inmediatamente (9-12). En RR, P3 tiene que esperar 4 segundos para su primera ejecución y luego es constantemente interrumpido, terminando en t=9.

3. ¿Cuál logra mayor Throughput y cuál mejor equidad?

Mayor Throughput: Ambos algoritmos tienen el mismo Throughput (0.1666...), ya que ambos terminan los 4 procesos exactamente en el mismo tiempo total (24 ciclos).

Mejor Equidad: RR logra una mejor equidad. A pesar de que los procesos terminan más tarde, todos los procesos (P1, P2, P3, P4) recibieron la CPU al menos una vez en los primeros 12 ciclos. FCFS (que es la base de RR) es justo en el reparto de turnos.

6. Ejercicio 6: SRTF vs. RR (\$q=2\$) con E/S Intercaladas

Objetivo: Comparar TAT y CS. ¿El SRTF realmente mejora el tiempo total?

1. Simulación (Secuencia de Ejecución)

- SRTF: Al ser expropiativo, los procesos con ráfagas cortas de retorno (P2 y P4) son priorizados inmediatamente.

| P1(0-1) | P2(1-4) | P4(4-6) | P2(6-7) | P4(7-10) | P1(10-14) | P3(14-17) | P1(17-19) | P3(19-22) |

- Round Robin (q=2): Flujo FIFO constante.

| P1(2) | P2(2) | P3(2) | P4(2) | P1(2) | P3(2) | P2(1) | P4(3) | P1(2) | ... (Termina en \$t=24\$).

2. Resultados de Métricas

Métrica Promedio	Round Robin	SRTF
Tiempo Medio Espera ($\$W\overline{T}\$$)	6.25	4.50
Tiempo Medio Retorno ($\$T\overline{A}\$$)	15.50	13.75
Cambios de Contexto (CS)	14	9

3. Respuesta a los Objetivos

- ¿Qué ocurre cuando los procesos cortos alternan mucho E/S?

El planificador SRTF los recompensa. Cada vez que P2 o P4 terminan su E/S y vuelven con una ráfaga de CPU corta, preemptan al proceso actual y salen rápidamente del sistema.

- ¿El planificador expropiativo realmente mejora el tiempo total?

Sí, SRTF reduce el TAT promedio. Aunque es más complejo de gestionar que RR, al priorizar la finalización de trabajos pequeños, SRTF desaloja los procesos rápidos del sistema antes, mejorando el rendimiento total.

7. Ejercicio 7: Prioridad Fija vs. Prioridad Dinámica (Aging)

Regla Dinámica: Cada tick que un proceso espera, su prioridad sube en 1 (máx. 1 = más alta).

1. Simulación y Evolución de Prioridades

- **Prioridad Fija:** P3(Pri 1) ejecuta t=2 a t=4. P2(Pri 2) ejecuta t=4 a t=8. P1 y P4 esperan hasta t=8.
- **Prioridad Dinámica (Aging):** P1 y P4 suben su prioridad mientras esperan.
 - o **t=1:** P1 espera (Pri 3 → 2).
 - o **t=2:** P1 espera (Pri 2 → 1). **P1 ahora es tan prioritario como P3.**
 - o **Resultado:** El envejecimiento permite que P1 o P4 se ejecuten mucho antes, interrumpiendo el flujo de P2, P3.

2. Resultados de la Simulación

Métrica Promedio	Prioridad Fija	Prioridad Dinámica (Aging)
$\$W\overline{T}\$$	6.75	4.25
$\$TA\overline{T}\$$	11.5	9.0

3. Respuesta a los Objetivos

- ¿Qué sistema evita mejor la inanición (starvation)?

El sistema de Prioridad Dinámica. Al implementar el envejecimiento, el proceso P1 (que de otra manera esperaría 6 ciclos) aumenta su prioridad y obtiene la CPU mucho antes, reduciendo su espera y evitando que los procesos de baja prioridad queden bloqueados indefinidamente.

8. Ejercicio 8: RR vs. Multinivel con Realimentación (MLFQ)

Reglas MLFQ: Q1(q=2), Q2(q=4), Q3(FCFS). Democión si se usa todo el quantum.

1. Simulación MLFQ (Progresión de Colas)

- **P3 (3s):** Ejecuta 2s en Q1 → Baja a Q2. Ejecuta 1s en Q2 → **Termina.** (Gana rápido).
- **P1 (9s):** Ejecuta 2s en Q1 → Baja a Q2. Ejecuta 4s en Q2 → Baja a Q3. Ejecuta 3s en Q3 → **Termina.** (Penalizado por ser largo).

2. Resultados de Métricas

Métrica Promedio	Round Robin	MLFQ
WT	10.75	8.25
RT	3.0	1.0

3. Respuesta a los Objetivos

- ¿Cuál logra mejor equilibrio entre respuesta y uso de CPU?

MLFQ logra el mejor equilibrio. Mantiene la rapidez de respuesta inicial (RT=1.0) al igual que RR, pero es más eficiente a largo plazo. Al mover los procesos largos (P1) a una cola FCFS (Q3), MLFQ reduce los cambios de contexto innecesarios y aumenta el rendimiento total.

9. Ejercicio 9: 2 Núcleos + Prioridades + Coste de Contexto

Escenario: 2 Cores. CS Costo=1. P1/P3 (Pri 1) en Core 0. P2/P4 (Pri 2/3) en Core 1.

1. Simulación (Timeline de Núcleos)

Ciclo(s)	Core 0 (P1, P3)	Core 1 (P2, P4)	Overhead (CS)
0 - 3	P1 (Q=3)	P2 (Q=3)	CS: 2
3 - 4	IDLE (CS=1)	P4 (Llega)	-
4 - 7	P3 (Q=3)	P2 (Q=3, termina)	CS: 2
7 - 8	IDLE (CS=1)	P4 (Q=3)	-
8 - 11	P1 (Q=3)	P4 (Q=3)	CS: 2
...	P1/P3 continúan	P4 continúa	-

2. Resultados y Evaluación

- Overhead por CS:** 4 ticks de CS x 1 tick perdido = **4 ciclos perdidos**.
- Utilización CPU:** (Tiempo Total de CPU) / (Tiempo Total - Overhead)
\$\\approx\$ **88%**.
- Conclusión:** El coste de contexto (\$CS=1\$) introduce una penalización notable. El tiempo se pierde en la gestión (overhead), no en la ejecución.

Esto justifica usar grandes quantums o afinidad de CPU para reducir los cambios.

10. Ejercicio 10: Caso Completo de Evaluación Final

Escenario: Prioridades, E/S, 2 Núcleos, CS=1, Q=2.

1. Simulación (Resumen)

- **RR:** Alto número de CS (costo alto), alta respuesta $\overline{R}$, pero tiempo total de simulación largo (el sistema espera la E/S de baja prioridad en los núcleos).
- **Prioridad Expropiativa:** P1 y P3 (Pri 1) dominan Core 0. P2 (Pri 2) y P4 (Pri 3) luchan en Core 1. Eficiencia alta, pero P4 sufre inanición.
- **MLFQ:** Combina la respuesta rápida con la eficiencia de los dos núcleos. Los procesos con E/S se benefician al volver rápido a la cola de alta prioridad.

2. Comparativa Final

Métrica	RR	Prioridades Expropiativas	MLFQ
Menor WT	Alto (15+)	Medio (8)	Medio (10)
Mejor RT (Interactividad)	Excelente (1.0)	Medio (3.0)	Excelente (1.5)
Menos CS (Eficiencia)	Bajo (Muchos)	Bueno (Medio)	Bueno (Medio)
Mayor Throughput	Bajo	Alto	Alto

3. Conclusión Final: Clasificación de Algoritmos

- **MLFQ** es el algoritmo más recomendado para este caso mixto (servidor/multitarea), ya que ofrece una buena respuesta inicial y gestiona las tareas pesadas con el mejor uso de ambos núcleos.